

Company: MSCI Inc.

Job Profile: Technology & Data Operations Intern

Offer Type: [Internship + performance based PPO]

Internship Stipend: 60,000/- per month

CTC: 14.67 LPA (13.2 base)

Location: Mumbai(Goregaon)/Pune

Process Date: **OA** - 11/07/2025, **Process** - 15/07/2025

Placement Process:

1. **OA** [No CGPA criteria AFAIK so everyone was eligible to sit for the OA]

The OA was a coding test, 75 minutes in duration. There were 2 coding/DSA problems and an SQL problem. The coding problems were both pretty basic and brute force would usually pass for all. I used Python. And I'd recommend using it if you haven't started with any language yet, because I think it's faster to type out :)

- **Problem 1:** There's a keypad. Each key takes a certain amount of time to press every time you press it. Input is a list of integer pairs like `[[0,2], [1,6], [3,8], [2, 4], ...]`. Here, `[0,2]` means 'a' took 2 seconds to be pressed and so on. So all letters (keys) are 0 indexed starting from 'a'. The output should be the letter of the key that had the maximum pressing time. I just created a pair `ret = [-inf, -inf]`, and traversed the input, let's call the current pair `cur`, and whenever `cur[1] > ret[1]`, set `ret = cur`. In the end just return the character associated with `ret[0]`.
- **Problem 2:** Input is a string. This string's various substrings contain multiple permutations of the word "programmer" as subsequences. The task is to find the number of letters between the first and last such substring. For example:

"rgtscmpxmrreoqwiqwrgmttpmrreo"

Here, the highlights are the first and last such substrings in the given string and answer is 6. (oqwiqw).

Many ways to solve this.

- **Brute Force:** Create a frequency map of the string "programmer". Now traverse the input string, and starting from each index, traverse ahead while building a temporary frequency map of this traversal. The first time this temp map becomes equal to our og freq map, store the final index of the current traversal. The last time that happens, store the first index of the final traversal. Then subtract both and add 1 and return.
- That's $O(n^2)$ but coding it up took like 10 minutes. Rather I could have just done the above starting from the first index first, then reverse the input string and do it from the

first index again, and store the final index of both, fix the reversed index, subtract, add 1.

- **Problem 3:** Given 2 tables, States, and Votes. The Votes table has the votername, state voted for. Return, for each state, a comma separated concatenated string of name of voter, and number of votes given, in the same cell.

Example output:

state	votes
Texas	John Doe x 2, Jane Doe x 8, ...
California	Tyler Durden x 6, Jack x 9, ...

Simple problem, but you have to remember the aggregate function's name that concatenates strings over the table. Logically it should be called something like `agg_str()` or `str_agg()` as it is in Postgres, but it is **GROUP_CONCAT()** in MySQL. This was given in the previous experiences but I guess I'd missed it. Don't make that mistake. Go through each and every single previous year experience from '24, '25 and this year's. They're the most efficient revision you'll get.

41 people were shortlisted. All solved at least the 2 coding problems. I'm guessing there was a time cut-off of 40-50 minutes max.

2. **Technical Interview L1** [41 people]

This round, for me, was just **DSA questions with a few introductory questions about my projects and internships**. As I entered, the first thing I was asked was the **full form of MSCI**. I got the last word wrong. He said "It's International! You should at least know this much bro!" and we laughed about it. Then he asked me what problems were there in the OA. I briefed him on shorter versions of the questions and explained how I solved them. He then apologized for jumping directly to questions and we **introduced ourselves to each other**. He was excited for the process and ended up taking a good 5 minutes introducing himself. I'd recommend developing the skill of ad libbing answers to these questions. Preparing "scripts" for intros, personal questions, even project questions kills your flow and the interview's vibe. It's meant to be a fun conversation. Don't make it boring or dull for the interviewer or yourself. I took my time, introduced myself, my internships and projects in depth for about 5-10 minutes.

He started asking DSA questions. The first one was **how to check if there are cycles in an undirected graph**. I know. Simple af. I didn't understand at first and misunderstood that it was directed and started explaining a topo sort approach (Kahn). He then said "it could be directed or undirected". And then my brain froze up. I got really confused and just could not think at all anymore. If that happens, drink some water or something to give your brain time to get back to normal. He saw me being confused af and he was like aight let's move on. He then asked **how to get the top view of a binary tree** (horizontal difference). Then **right/left view** (bfs). Then a matrix problem, a spin on the "**number of ways to get from top left to bottom right cell of a matrix**" problem (simple 2d dp). So all the dsa problems were very basic. When you get something wrong, be graceful about it. I was very surprised that I got selected for the next round.

He then asked me a little about my projects, and **how I handled authentication** in them, and **how to do it without using a third party service**. I replied “using JWT” and that building a middleware that verifies the JWT header was a viable solution. He **inquired a bit about my knowledge of JWT**. I told him whatever I knew about it.

He asked me whether I know **what flexbox is**. I described its usecase, and compared it to css grids which came later but are much cleaner for 2d layouts. I added that flex animates while grid doesn't. But I now think that wasn't completely true. He asked me **whether I've ever written pure CSS**. I said even while using bootstrap (which I did in an internship), we have to write pure CSS to style low level bootstrap components, so yes I have.

He then asked me **why I want to pursue a career in software when I am from the EXTC branch**. I just explained to him how I'd found the EXTC syllabus more interesting and why, before I took admission. But actually studying something and building projects based on that knowledge is very different from what its syllabus looks like on a table of contents. I frequently built IOT projects in my first year and worked with various microcontrollers like the ESP and STM32; Over time, I found myself more drawn to the coding side — not just programming the microcontrollers, but also building client-facing software for those systems. That naturally pulled me deeper into core CS subjects, which I studied alongside my coursework. It wasn't an easy switch but I'm glad I made it, and now I have knowledge on both ends of the technology spectrum.

My interview went like a very interesting conversation you have with a friend. A lot of it was just him talking about his work at MSCI and me asking questions, or me talking about my internships and projects, both of which he found interesting. I'd frozen up on one question and still I was called for the next interview. Some people had answered all of theirs right and still weren't. It's a combination of the panel and your conversation skills. To be safe, get every question right. But if you don't, be graceful about your wrong answers. Make a joke about it. Keep the interview light and always keep a classy, genuine smile up throughout.

People were being called for the next round one by one as their reviews were coming in. I don't know the exact figure but I don't think more than 30 people got through.

3. **Technical Interview L2** [I think around 25-30 people probably got through]

Always walk in with a bright smile and a “how are you”. Don't be a drag and just walk in and sit. Shake their hand. Lighten up the environment. Ask them about their day. The first thing I was asked this time was again the full form of MSCI. I said Morgan Stanley Capital International and was promptly shut down. “We don't have a full form”. “Always the trick questions”, I lamented jokingly and we laughed about it. He asked about my day and who my previous interviewer was. Then he opened his laptop and on it, my resume. (By the way, this round's interviews were taken by the Executive Directors of MSCI.)

He began by asking me to introduce myself. Then he asked me which project I would like to start with. I went with the first real-world project I'd ever built: Tetrace. He opened the github repo. I'd drafted a nice ReadMe which he read through while I described it. I told him how I'd been part of the backend team of 2, and that Tetrace is a location based treasure hunt, where clues are provided to players and they've to reach the locations, and there are powercards yada yada (I'm sure y'all know what it is). He

asked me what db we used. We used Firebase RTDB for real time streaming, and Firestore for static data. He asked me to describe the schema and the whole architecture. I took my time, about 5 minutes. He asked me why RTDB. I explained that team state, powercard timestamps, and location data are better stored in the RTDB. He asked me why. That data requires real time sync and it's a real time game, it needs lower latency; also rtdb has some offline support so I plugged that in. **"So the client side application would read directly from the rtdb for some features?"**. Yep. **"Isn't that unsafe?"**. It seems like there would be data inconsistencies, but the client was not allowed to write to the db, only read from it; and the reads would only be on non-critical data.

"Suppose I apply the freeze on a team, and at the approximately same time someone else applies the freeze on the same team. What happens?". I covered a piece of paper in the JSON schema of the application, the model of the powercard states, and the controller for powercard usage. I explained that using the async-mutex npm module, we create mutex states of each team stored in a js object in-memory. When the request to freeze team 007 comes from team 069, and it reaches the controller, Mutex.acquire() is called on the state of that team. When the request to freeze 007 then comes from team 420, if the lock is not released yet, the respective error object is thrown for the later request. He asked me to write the full code and I obliged. He asked which other tech on my resume has the mutex functionality. After a bit of thought, and mentioning Java for its semaphore functionality, I realized he was talking about golang.

"Very good. Now, what happens if there is a loading error and I click a powercard twice? How much of my balance will be deducted?". I explained that we couldn't handle concurrent requests in-game because of our lack of knowledge back then. **"How would you do it?"**. I said I could create a similar object of lock states for the requestor. He found that to be smart but asked me to **think further for a better solution**. I said I could apply high precision rate limiting, or enforce time gaps between requests. **"Gaps for erroneous requests can also vary"**. DB-level transaction control is offered by firebase. He questioned its thoroughness. I conceded I didn't know any more solutions. He revealed assigning UUIDs for each request from each user is the best way. Requests with the same UUID are erroneous.

He asked a **few questions about the stack and what exactly my part was in the backend**. I said I was doing the C of the MVC, aside from the error handling and frontend interfacing. We moved on to my next project, which was a pure Java based low latency stock exchange engine core, with a spring boot wrapper for the communication layer.

I explained to him the full architecture of the entire system. He asked me to **jot down the db schema**. I did. He asked me some questions about the schema. I answered them, but while I did I realized he was going deep into the database side of this project which was not the purpose of the project at all. I explained to him the purpose of the project was to replicate the structure and architecture of a real stock market engine core and use an object oriented approach for the prototype while also keeping it as low latency as possible. He then started asking me questions about the business logic once he realized the persistence layer mustn't be as well thought out whereby.

So no persistence? How do you process this very important data?

In-memory. Using relevant data structures. So all the DSA knowledge doesn't go to waste.

What data structures did you use?

So there will be 2 order books for each company, buy and sell. For price-time priority, each limit price will be stored in a sorted hashmap, specifically a TreeMap. And the orders at each price will be stored in a doubly linked list.

Why TreeMap? What's a TreeMap?

And he asked about its abstracted implementation. I explained the whole Map, Hashmap, SortedMap, TreeMap hierarchy, and it's built on Red-Black trees.

Why a doubly linked list for orders? Couldn't you use a-

Yes I initially used a queue for it, but when I benchmarked the engine, order cancellation was a bottleneck. DLLs have faster removal.

Sketch the OOP model

I wrote down a summarizing code of how the whole system worked. While answering all of the above questions too, I'd been sketching detailed figures while explaining everything so he is on the same page the whole time.

Then, based on the rough ER diagram I'd drawn prior, he asked me a few questions. To simplify the only sorta challenging one:

Tables: User (uid, uname, balance, holdings), Trades (tradeId, sellerId, buyerId, volume, price).

Return the number of times I have sold and bought a set of stocks.

I first misunderstood it such that I have to return that for every user. So I wrote a subquery that left joins User with Trades on uid = sellerid, and returns uname, count(tradeid) as sells. And I explained that a parent query can return the name, sells and buys. He explained he only wanted it for one user, which I said would just be a join and where clause for both buy and sell and he agreed. He said there's a much easier way to do this and that there's no time now but I should think about it. I said every time I've ever used a subquery, there's turned out to be much easier ways to do that task that made me facepalm. That made him laugh.

He then asked me about my internships, and asked me why I've only done ones in front-end roles. I said I got them based on the level of skills and knowledge I already had at the time, and I was much better at frontend technologies at the time of beginning them. He asked what the biggest obstacles had been for me on the technical and the professional side of my projects or internships, and how I'd overcome them.

He asked me if I had any questions for him. I asked him how long he's been with the company and how his experience has been. He didn't seem very interested in answering questions like these so I switched it up and asked him what type of work could I expect to be assigned if I were in his team. He said he has no idea what type of work is even assigned to immediate interns and we laughed a bit about that. The whole interview went perfectly. I think it could be the longest L2 interview of the day – it lasted for about 1.5 hours.

Around 9 candidates were selected for the HR round. I was probably the 4th. I was called in as soon as I came out of the interview.

3. HR Interview [9 candidates]

I was asked about my day till then, and how the rest of my interviews had been. I'll just list all the questions then a bit about my approach for answering them.

- About yourself and your family. Your hobbies.
- About yourself professionally.
- So many technologies in your project, why not stick to one.
- Have you worked in a team?
- Prefer working in a team or solo.
- Why xyz internship
- Extra curricular activities in college life?
- Our work hours can be off putting to some. Can you handle them?
- Prefer Mumbai or Pune?
- You trade, so will you be ok with the company telling you to stop because of our rules?
- What have been your biggest strengths and weaknesses throughout your professional life.

Be genuine, and explain with thorough examples. Your aim is to convince them of your answer's sincerity, not just tell them your answer.

Again, I can't stress this enough, smile.

5 people were offered the internship in the end.

Useful Tips:

OA's: Brute force almost *always* works. And every minute is valuable. Code up the brute force, see if it runs. Probably will. And don't get intimidated by how many leet problems other people have done. On-campus OA's are easy. Don't need to be a specialist on cf to solve them. But if you have:

- A year before placements: do give as many cf contests as you can. They teach you how to think. I;m not saying rank push. But get yourself to a spot where you can solve div 2 A and B always, and C at least half the time.
- Or if you have very few months before placements and you don't have a single leet problem. Do top interview 150, then some 10-20 random medium/hards, and you're good to go tbh. But do all of them like they haven't been solved yet. Think till you either come up with the solution or are weeping profusely. I could clear 2 out of 3 OA's with that prep level, but your batch could be stronger than ours so work relative to that.

Some life advice no one asked for:

- **If I could go back in time I'd be more consistent for longer durations in my DSA journey, and wouldn't stop giving CF contests for hackathons. I didn't evaluate my strengths correctly and let peer pressure guide me to a path that wasn't made for me. Don't make that mistake. You won't get a chance to try everything, so only let your judgement guide you. Whether you wanna win hacks, or contribute to open source, or do internships, or build creative projects, or be a competitive coder. Give your all at whatever you pick. Don't do what your friends are doing. Pick friends that are doing what you want to do. Oh and don't be a phoney. Only pick the CS journey if you're passionate about it. If you're not and you still have to pick it, then convince yourself that you're passionate about it. ^_(\ツ)_/^-**

Technical Interviews: Doesn't happen often but being a jack of all trades actually helped me in this round. I was able to answer any questions that came up, in terms of tech stack. I was asked a lot of questions about the various technologies I'd used, from common ones like React, Springboot, Node to relatively eccentric ones like Vue and Go. If they aren't in your resume, you won't be asked about them. But if they are, and you can answer the questions, that leaves an impression because it shows your passion.

For interviews my advice will always be to stop preparing answers. Ad lib. Improvise. Read books, watch shows, interact more, touch grass, IDK whatever you need to do. But hey, what do I know? Whatever suits you in your opinion is probably better for you than what I can tell :) Also sorry for making this IE this long goddamn. And also for the shite formatting lol.

Lastly, don't be stressed about placements. It's better to feel less happy when you're finally placed, than to feel more sad every time you're rejected. So walk into your first test assuming it'll take 50 more of these before you're finally placed. Make these tests/interviews feel like a chore, not a burden. Also get CTC out of your mind. You young as hell. Think about the work you'll get to do, that's way more important.

All the best y'all!

Feel free to [connect and ask questions](#); Linkedin haters, hmu on whatsapp: 7718965798

Or let's play a [game of chess](#)!

Misc. (for EXTC peeps): This company does not have any implicit or explicit bias against us. I'm not saying others do, but I can say for sure that this one doesn't. It's a good opportunity. Just make sure they get to know you're truly passionate about CS. Reach out and connect with me any time, and I will help you out to the best of my capabilities. Some companies don't allow EXTC, some don't allow pointers less than 8. But just know that it's all the same when you finally get a shot. Good luck guys.